



CP Bootcamp 2026 — Day 13

Theme: Trees + Binary Search Tree Foundation

Main Goal:

আজ থেকে তুমি linear data structure থেকে **hierarchical data structure**-এ যাবে।

এতদিন:

Array

[1][2][3][4]

Linked List

1 → 2 → 3

আজ:

Tree

```
  1
 / \
2   3
```

আজকের focus:

Tree Structure

↓

Node Relationship

↓

Tree Traversal

↓

Binary Tree

↓

Binary Search Tree

Estimated Time: 6–7 hours



Day 13 Chapter Map

Day 13

- Chapter 1 – Tree Mental Model
- Chapter 2 – Tree Terminology
- Chapter 3 – Binary Tree Concept
- Chapter 4 – Tree Traversal (DFS)
- Chapter 5 – Level Order Traversal (BFS)
- Chapter 6 – Binary Search Tree (BST)
- Chapter 7 – BST Operations
- Chapter 8 – Tree Problem Patterns
- Chapter 9 – Common Bugs & Edge Cases
- Chapter 10 – Day 13 Assignment & Practice

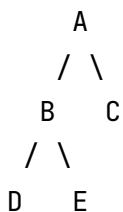
Chapter 1 — Tree Mental Model

Tree কী?

Tree হলো:

একটি hierarchical data structure যেখানে node-গুলো parent-child relationship দিয়ে connected থাকে।

Example:



এখানে:

A → Parent

B,C → Children

D,E → Grandchildren

Real Life Example

Folder structure:

Computer

└ Documents

| └ File1

| └ File2

└ Pictures

এটাও একটি tree।

Tree vs Linked List

Linked List:

10 → 20 → 30

একটা direction।

Tree:

```
    10
   /  \
  20   30
```

একটি node থেকে multiple child যেতে পারে।

Chapter 2 — Tree Terminology

Tree বুঝতে কিছু শব্দ জানা দরকার।

Node

Tree-এর প্রতিটি element।

Example:

10

↑

Node

Root

সবচেয়ে উপরের node।

Example:

```
  1
 / \
2   3
```

Root:

1

Parent

যে node-এর নিচে child আছে।

Example:

$$\begin{array}{c} 1 \\ / \\ 2 \end{array}$$

1 হলো:

Parent

Child

যে node parent-এর নিচে থাকে।

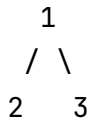
2 হলো:

Child

Leaf Node

যার কোনো child নেই।

Example:



Leaf:

2

3

Edge

দুই node-এর connection।

Example:

1 — 2

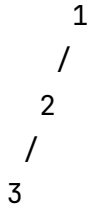
এখানে edge:

1-2

Height

Root থেকে সবচেয়ে দূরের leaf পর্যন্ত distance।

Example:



Height:

2

Depth

একটি node root থেকে কত দূরে।

Example:



2-এর depth:

1

Chapter 3 — Binary Tree Concept

Binary Tree কী?

যে tree-এর প্রতিটি node-এর maximum দুইটি child থাকে।

Child:

Left

Right

Example:

```
      10
     /  \
    5    20
```

Node Structure in C:

```
struct Node
{
    int data;

    struct Node *left;

    struct Node *right;
};
```

একটি node:

```
    data
    |
left pointer
    |
right pointer
```

Create Node

```
struct Node* createNode(int value)
{
    struct Node* newNode;

    newNode = malloc(sizeof(struct Node));

    newNode->data=value;

    newNode->left=NULL;

    newNode->right=NULL;

    return newNode;
}
```

Creating Tree Manually

Example:

```
    1
   / \
  2   3
```

Code:

```
root=createNode(1);

root→left=createNode(2);

root→right=createNode(3);
```

Chapter 4 — Tree Traversal (DFS)

Tree-এর সব node visit করার process:

Traversal

DFS:

Depth First Search

মানে:

আগে গভীরে যাওয়া।

DFS-এর তিনটি main type:

1. Preorder
2. Inorder
3. Postorder

Tree:

```
    1
   / \
  2   3
```

1. Preorder

Order:

Root

Left

Right

Shortcut:

NLR

Output:

1 2 3

Code:

```
void preorder(struct Node* root)
{
    if(root==NULL)
        return;

    printf("%d ",root->data);

    preorder(root->left);

    preorder(root->right);
}
```

2. Inorder

Order:

Left

Root

Right

Shortcut:

LNR

Output:

2 1 3

Code:

```
void inorder(struct Node* root)
{
    if(root==NULL)
        return;

    inorder(root->left);

    printf("%d ",root->data);

    inorder(root->right);
}
```

3. Postorder

Order:

Left

Right

Root

Shortcut:

LRN

Output:

2 3 1

Code:

```
void postorder(struct Node* root)
{
    if(root==NULL)
        return;

    postorder(root->left);

    postorder(root->right);

    printf("%d ",root->data);
}
```

Traversal Memory Trick

Tree:



Preorder

আগে root:

A B C

Inorder

Root মাঝখানে:

B A C

Postorder

শেষে root:

B C A

Chapter 5 — Level Order Traversal (BFS)

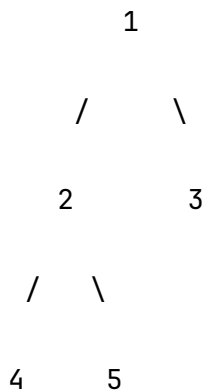
DFS:

Depth first

BFS:

Level first

Example:



Level order:

1 2 3 4 5

BFS uses:

Queue

কারণ:

First node আগে process হবে।

Algorithm:

Push root

↓

While queue not empty

↓

Remove front

↓

Add children

Pseudo:

```
queue.push(root)
```

```
while queue not empty:
```

```
    node = queue.pop()
```

```
    print node
```

```
    push left child
```

```
    push right child
```

Chapter 6 — Binary Search Tree (BST)

BST কী?

Binary Tree-এর special type।

Rule:

For every node:

Left side

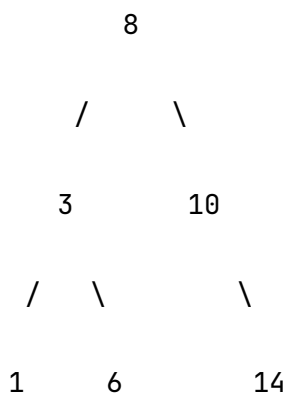
<

Root

<

Right side

Example:



Check:

Left:

$3 < 8$

Right:

$10 > 8$

Correct BST!

Why BST?

Search faster করার জন্য।

Normal Binary Tree:

Need:

$O(n)$

BST:

Average:

$O(\log n)$

Chapter 7 — BST Operations

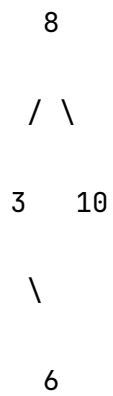
1. Search

Example:

Find:

6

Tree:



Start:

8

Compare:

$6 < 8$

Go left!

3:

$6 > 3$

Go right!

Found:

6

Code:

```
struct Node* search(struct Node* root,int target)
{
    if(root==NULL || root->data==target)
        return root;

    if(target < root->data)
        return search(root->left,target);

    return search(root->right,target);
}
```

2. Insert

Rule:

Same as search.

Example:

Insert:

7

Find correct position!

Code:

```
struct Node* insert(struct Node* root,int value)
{
    if(root==NULL)
        return createNode(value);

    if(value < root->data)
        root->left=insert(root->left,value);

    else
        root->right=insert(root->right,value);

    return root;
}
```

3. Minimum Value

BST-তে:

সবচেয়ে ছোট value:

সবচেয়ে বাম পাশে

Example:

8
/
3
/
1

Minimum:

1

Code:

```
int minimum(struct Node* root)
{
    while(root->left != NULL)
    {
        root = root->left;
    }

    return root->data;
}
```

Chapter 8 — Tree Problem Patterns

Pattern 1 — Traversal

Problem:

"Visit all nodes"

Think:

DFS/BFS

Pattern 2 — Height

Need:

Maximum depth

Pattern:

Recursive
+
`Max(left,right)`

Formula:

`height(node)`

=

`1 + max(left,height,right,height)`

Pattern 3 — Search

BST হলে:

Compare

Move Left/Right

Pattern 4 — Level Problems

Example:

"Print nodes level by level"

Think:

Queue + BFS

Chapter 9 — Common Bugs & Edge Cases

Bug 1 — Empty Tree

Always check:

```
if(root==NULL)
```

Bug 2 — Wrong Traversal Order

Confuse:

Preorder

Inorder

Postorder

Bug 3 — BST Rule Break

Wrong:

5

/

8

Left child বড়।

BST না।

Bug 4 — Forget Return in Recursive Function

Wrong:

```
insert(root→left,value);
```

Need:

```
root→left=insert(root→left,value);
```

Bug 5 — Unbalanced BST

Example:

Insert:

```
1
2
3
4
5
```

Tree:

1

\

2

\

3

Linked list-এর মতো হয়ে যায়।

Complexity:

$O(n)$

Chapter 10 — Day 13 Assignment

Part A — Binary Tree

Implement:

1. Create Node
2. Preorder traversal
3. Inorder traversal
4. Postorder traversal
5. Count nodes
6. Find height

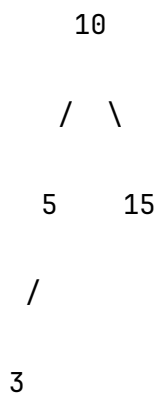
Part B — BST

Implement:

1. Insert
2. Search
3. Find minimum
4. Find maximum
5. Inorder traversal

Part C — Dry Run

Tree:



Find:

Preorder :

Inorder :

Postorder :

Height :

Part D — Pattern Recognition

Problem:

Level by level print করতে হবে।

Pattern:

?

Problem:

Sorted order-এ BST values print করতে হবে।

Pattern:

?

Problem:

Middle node of linked list.

Pattern:

?



Glossary Update

Add:

Tree

Node

Root

Parent

Child

Leaf

Edge

Height

Depth

Binary Tree

Traversal

DFS

BFS

Preorder

Inorder

Postorder

Binary Search Tree

BST

Balanced Tree

—



Pattern Library Update

Add:

`## Tree Traversal Pattern`

Recognition:

`Need to visit all tree nodes.`

DFS:

Recursion

BFS:

Queue

Types:

Preorder

Inorder

Postorder

BST Search Pattern

Recognition:

Data follows:

Left < Root < Right

Mental Model:

Compare

↓

Move Left/Right

↓

Repeat

Day 13 Completion Checklist

- ☐ Tree concept বুঝি
- ☐ Node relationship বুঝি
- ☐ Root/Leaf বুঝি
- ☐ Binary Tree বুঝি
- ☐ DFS traversal লিখতে পারি
- ☐ Preorder বুঝি
- ☐ Inorder বুঝি
- ☐ Postorder বুঝি
- ☐ BFS বুঝি
- ☐ Queue দিয়ে level order করতে পারি
- ☐ BST rule বুঝি
- ☐ BST insert করতে পারি
- ☐ BST search করতে পারি
- ☐ Tree height বের করতে পারি



Day 13 Final Mental Model

Problem



Need hierarchy?



Tree



Need visit all?



Traversal



Need fast search?



BST



Need level processing?



Queue + BFS

Day 13 শেষে তোমার foundation:

Day 1-5
Patterns

+

Day 6
Functions

+

Day 7
Searching

+

Day 8
Sorting

+

Day 9
Optimization

+

Day 10
Recursion

+

Day 11
Stack Queue

+

Day 12
Linked List Hashing

+

Day 13

Trees

পরের:

Day 14 — Graph Foundation + BFS/DFS হবে। 